

1장 자료구조와 알고리즘

자료구조란?

생활속의 다양한 사물과 관련 자료구조

리스트 : 배달할 선물 목록

스택 : 접시나 물건 쌓기

큐 : 매표소 줄서기

그래프 : 지하철 노선도

트리 : 직장의 조직도

★트리는 circulation이 없고, 그래프는 circulation이 있을 수 있다.

★그래프≧트리

자료구조의 분류

선형 자료구조 : 자료들을 일렬로 나열. 순서가 중요!

스택, 큐, 데크(deque; double ended queue), list 등

비선형 자료구조 : 선형으로 나열하기 어려운 복잡한 관계의 자료들을 표현

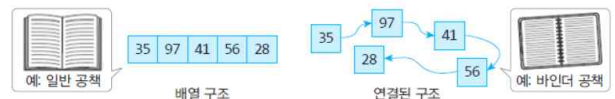
트리, 그래프, 집합(set) 등

자료구조의 구현방법

배열구조 - Array

모든 데이터가 **인접한 메모리** 공간에 저장

크기가 고정된 일반 공책과 비슷. 탐색(검색)은 쉽지만, **가득차면 더 이상 저장 불가**



연결된 구조 - Linked List

흩어져 있는 데이터들을 연결하여 하나로 관리

바인더 공책과 비슷. 위치를 바꾸거나 쉽게 추가/삭제할 수 있어 **유연하게** 사용 가능하지만 **관리가 복잡**

자료구조와 알고리즘

프로그램 = 자료구조 + 알고리즘

알고리즘이란?

: 주어진 문제를 해결하기 위한 단계적인 절차

: 프로그래밍 언어와 상관없이 문제 해결 절차를 나타내는 명령어의 집합

프로그래밍을 위해 작성된 모든 명령어의 집합이 알고리즘이 되는 것은 아님. 6개 조건 만족해야 함.

1. 입력 : 0개 이상의 잘 정의된 입력이 존재해야 함. - 난수는 입력 필요없음.
2. 출력 : 1개 이상의 명확히 정의된 출력값이 존재하여야 한다.
3. 명확성 : 각 명령어의 의미는 모호하지 않고 명확해야 한다.
4. 유한성 : 한정된 수의 단계 후에는 **반드시 종료**되어야 한다.
5. 유효성 : 각 명령어들은 실행 가능한 연산이어야 한다.
6. 언어 독립성 : 특정한 프로그래밍 언어와 상관이 없어야 한다.

알고리즘의 기술 방법

자연어, 흐름도(flow-chart), Pseudo-code, 특정 프로그래밍 언어 등...

자연어 : 표현이 자유로움. 단어가 정확하지 않으면 모호해질 우려 있음.

Flow-Chart(흐름도) : 직관적이고 이해가 쉬움. 복잡한 알고리즘을 표현 시 복잡해짐.

특정 프로그래밍 언어 : 알고리즘을 가장 정확히 기술 가능. 구체적인 사항이 핵심 내용을 방해할 수 있음.

유사코드(Pseudo-code) : 가장 많이 사용. 체계적이지만 프밍언어보다는 덜 엄격. 핵심적인 내용에 집중 가능

자료형

데이터의 종류 : "데이터의 종류"

정수, 실수, 문자열 등이 기초적인 자료형의 예

데이터 + 연산이 동시에 정의가 되어 있어야 함.



추상 데이터 타입(ADT)

데이터 타입을 추상적(수학적)으로 정의한 것

추상화 : 데이터 타입에서 필수적이고 중요한 특징만을 골라서 단순화 한 것.

데이터나 연산이 **무엇**인가는 정의. 데이터나 연산을 어떻게 컴퓨터 상에서 구현할 것인지는 정의되지 않음.

데이터 : 추상 데이터 타입에 속하는 데이터가 정의됨.

연산 : 이들 객체들 사이의 연산이 정의됨. 외부로 연결하는 인터페이스의 역할을 함!

알고리즘의 성능 분석

알고리즘 효율성의 중요성 : 빨라야 함!

시간 효율성. 공간 효율성

수행시간 측정 : 각 알고리즘을 실제로 구현하여 수행 시간을 측정

실행시간 = 종료시간 - 현재시간

C언어에서 clock() 함수 사용

시스템의 현재 시간 반환(CLOCKS_PER_SEC 단위)

수행시간 = (stop - start) / CLOCKS_PER_SEC

문제점 : 반드시 **구현**이 되어야 함

똑같은 하드웨어, 소프트웨어 환경 사용

동일한 프로그래밍 언어(컴파일 vs 인터프리터)

평가에 사용한 데이터에 한해서만 유효

□ C언어에서 사용하는 두 가지 시간 측정 함수

▣ clock(): CLOCKS_PER_SEC로 나뉘어서 사용

▣ time(NULL): difftime() 함수 사용

방법 #1	방법 #2
<pre>#include <time.h> start = clock(); ... stop = clock(); double duration = (double)(stop - start) / CLOCKS_PER_SEC;</pre>	<pre>#include <time.h> start = time(NULL); ... stop = time(NULL); double duration = (double) difftime(stop, start);</pre>

복잡도 분석 : 직접 구현하지 않고 알고리즘이 수행하는 **연산 횟수**를 계산

직접 구현하지 않고도 모든 입력 고려할 수 있음.

HW나 SW와 관련없이 알고리즘의 효율성을 평가할 수 있음.

시간공간도 : 알고리즘의 수행시간 분석

공간복잡도 : 알고리즘의 사용공간 분석

>> 통상적으로는 **시간복잡도 분석**을 의미함.

더 중요!!

알고리즘 A	알고리즘 B	알고리즘 C
sum $\leftarrow n * n$;	for $i \leftarrow 1$ to n do sum $\leftarrow$ sum + n ;	for $i \leftarrow 1$ to n do for $j \leftarrow 1$ to n do sum $\leftarrow$ sum + 1;

	알고리즘 A	알고리즘 B	알고리즘 C
대입연산	1	n	$n * n$
덧셈연산		n	$n * n$
곱셈연산	1		
나눗셈연산			
전체연산수	2	$2n$	$2n^2$

복잡도의 점근적 표기(Asymptotic notation)

n 이 커질수록 계수의 영향력이 미미해지고, 최고차항을 제외한 나머지 항의 효과도 급속히 줄어든다.

점근적 : 가장 큰 영향을 주는 항만 계산한다는 의미

정확히 몇 번인가가 궁금한 것이 아님. 증가 속도에만 관심.

Big-O notation 빅오 표기법

$O(g(n))$

증가 속도가 $g(n)$ 과 같거나 낮은 모든 복잡도 함수

$2n+1$ 이면 $O(n)$ 에 속함.

처리 시간의 **상한(upper bound)**을 의미

$O(n^2)$ 은 아무리 늦어도 n^2 안에 끝남

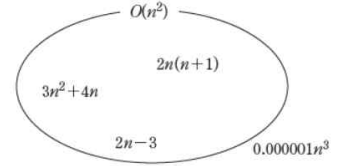
$$3n^2 + 4n \in O(n^2)$$

$$2n - 3 \in O(n^2)$$

$$2n(n+1) \in O(n^2)$$

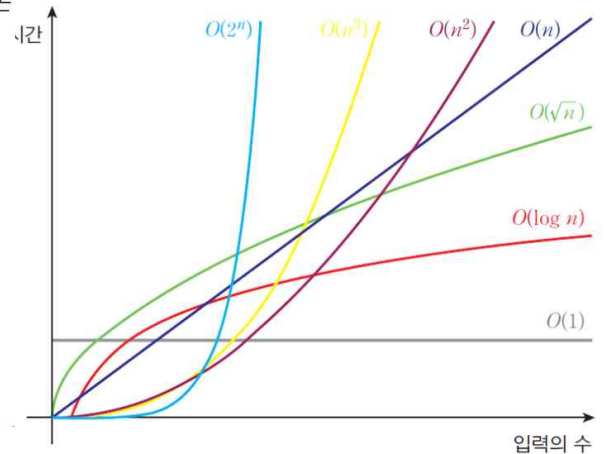
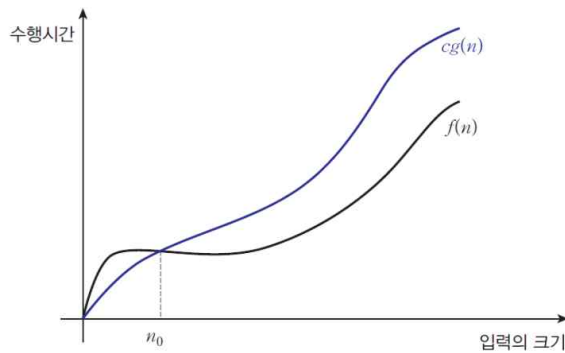
$$0.000001n^3 \notin O(n^2)$$

$$1000^n \in O(n!)$$



두 개의 함수 $f(n)$ 과 $g(n)$ 이 주어졌을 때,

모든 $n \geq n_0$ 에 대하여 $|f(n)| \leq c|g(n)|$ 을 만족하는 2개의 상수 c 와 n_0 가 존재하면 $f(n) = O(g(n))$ 이다.



시간복잡도	n					
	1	2	4	8	16	32
1	1	1	1	1	1	1
$\log n$	0	1	2	3	4	5
n	1	2	4	8	16	32
$n \log n$	0	2	8	24	64	160
n^2	1	4	16	64	256	1024
n^3	1	8	64	512	4096	32768
2^n	2	4	16	256	65536	4294967296
$n!$	1	2	24	40326	20922789888000	26313×10^{23}

- $O(1)$: 상수형
- $O(\log n)$: 로그형
- $O(n)$: 선형
- $O(n \log n)$: 선형로그형
- $O(n^2)$: 2차형
- $O(n^3)$: 3차형
- $O(2^n)$: 지수형
- $O(n!)$: 팩토리얼형

Big-Ω notation 빅오메가 표기법

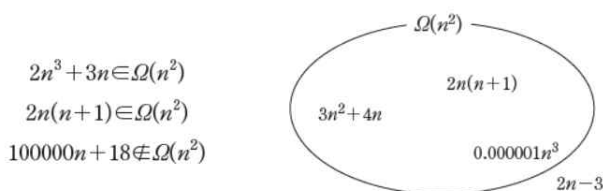
$\Omega(g(n))$

증가 속도가 $g(n)$ 과 같거나 높은 모든 복잡도 함수

$\Omega(n^2)$ 은 아무리 빨리 처리하더라도 n^2 이상의 시간은 걸림

처리 시간의 **하한(lower bound)**을 의미

모든 $n \geq n_0$ 에 대하여 $|f(n)| \geq c|g(n)|$ 을 만족하는 2개의 상수 c 와 n_0 가 존재하면 $f(n) = \Omega(g(n))$ 이다.



$$2n^3 + 3n \in \Omega(n^2)$$

$$2n(n+1) \in \Omega(n^2)$$

$$100000n + 18 \in \Omega(n^2)$$

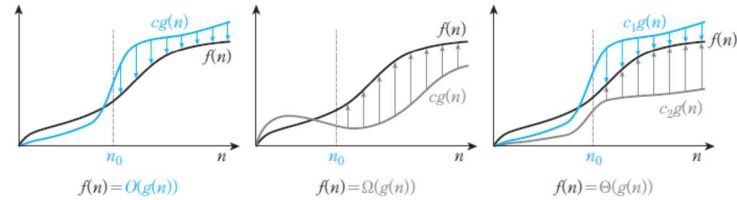
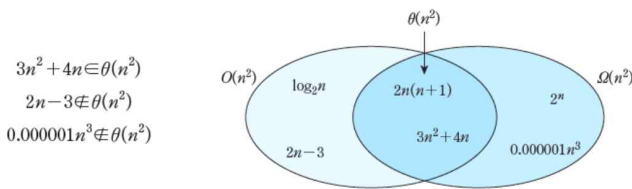
Big-θ notation 빅세타 표기법

$\theta(g(n))$

증가 속도가 $g(n)$ 과 같은 모든 복잡도 함수

상한(upper bound)인 동시에 하한(lower bound)

- 모든 $n \geq n_0$ 에 대하여 $c_1|g(n)| \leq |f(n)| \leq c_2|g(n)|$ 을 만족하는 3개의 상수 c_1, c_2 와 n_0 가 존재하면 $f(n) = \theta(g(n))$ 이다.



최선, 평균, 최악의 경우

똑같은 알고리즘도 입력 집합에 따라 수행 시간이 다를 수 있음

최선의 경우 : 수행 시간이 가장 빠른 경우 - 의미가 없는 경우가 많음

평균의 경우 : 수행 시간이 평균적인 경우 - 계산하기가 상당히 어려움

최악의 경우 : 수행 시간이 가장 늦은 경우 - 가장 널리 사용됨. 계산하기 쉽고 응용에 따라 중요한 의미를 가짐.

2장 순환(재귀; Recursion)

순환(재귀, Recursion)이란?

알고리즘이나 함수가 수행 도중에 자기 자신을 다시 호출하여 문제를 해결하는 기법. aka 재귀

분할정복(Divide and conquer) : 주어진 문제를 더 작은 동일한 문제로 분해하여 해결하는 방법

예제1 팩토리얼

그 수보다 작거나 같은 모든 양의 정수의 곱

$$n! = n * (n-1) * (n-2) * \dots * 2 * 1$$

방법 1) 함수를 여러개 만들기

함수를 여러개 만들어서 하위 단계를 각각의 함수를 호출해서 실행

-> 비효율적이다!

```
int factorial(int n)
{
    if( n <= 1 ) return(1);
    else return (n * factorial_n1(n-1));
}
```

```
int factorial_n1(int n)
{
    if( n <= 1 ) return(1);
    else return (n * factorial_n2(n-2));
}
```

방법 2) factorial() 함수 하나로 순환적 호출

factorial() 함수를 하나 만들어서 매개변수만 다르게 하여 순환적(재귀적)으로 호출

순환 호출의 내부적인 구현

시스템 스택 : 함수 호출을 관리하기 위한 스택.

활성화 레코드(Activation Record)가 저장됨.

활성(화) 레코드 : 함수 실행 중에 다른 함수를 호출하면 실행 중이던 함수의 context(문맥) 정보(지역변수, 파라미터, 복귀 주소, 반환값)를 활성 레코드에 저장하여 보관.

지역변수 : 함수 내에서 선언된 지역변수를 저장하는 곳

파라미터 : 호출 함수가 피호출 함수에게 넘기고자 하는 값

반환값 : 피호출 함수가 호출 함수에게 반환하는 값

복귀주소 : 함수가 호출된 곳의 다음 주소

- 200줄에서 204 줄을 호출하고, inst의 길이가 4바이트이면 다음 명령은 204가 복귀 주소가 됨.

```
int factorial(int n)
{
    if( n <= 1 ) return(1);
    else return (n * factorial(n-1));
}
```

순환 알고리즘의 구조

순환 호출을 하는 부분 + 순환 호출을 멈추는 부분

만약 Exit Condition이 없다면 시스템 스택을 다 써버리게 됨. 오류 발생 >> 반드시 종결조건이 포함되어야 함!

```
int factorial(int n)
{
    if( n <= 1 ) return 1;
    else return n * factorial(n-1);
}
```

← 순환을 멈추는 부분

← 순환 호출을 하는 부분

순환(재귀) vs 반복

★ 대부분의 순환(재귀)은 반복으로 바꾸어 작성할 수 있다★

역) 반복은 순환으로 바꿀 수 없다!

순환 : 높은 가독성 및 프로그래밍의 용이성
함수 호출하는 데 오버헤드가 듬.

>> 코드가 짧다!

>> 시스템 코딩에서 사용X. context switch 많이 발생

반복 : 속도가 빠름
본질적으로 순환적인 문제에는 부적합.

Tail Recursion은 반복으로 쉽게 변경 가능! but, Head Recursion이나 Multi Recursion은 변경 어려움.

예제 1) 팩토리얼의 반복적 구현

$$n! = \begin{cases} 1 & n = 1 \\ n * (n-1) * (n-2) * \dots * 1 & n \geq 2 \end{cases}$$

```
int factorial_iter(int n)
{
    int i, result = 1;
    for(i=1; i<=n; i++)
        result = result * i;
    return(result);
}
```

시간복잡도 : 순환 : O(n)

반복 : O(n)

>> 시간복잡도는 둘 다 같다! but, 순환은 추가 메모리 필요 + 스택처리 오버헤드 有. 시간이 더 오래 걸림.

예제 2) 거듭제곱 값 계산

x^n 을 구하자!

반복 ----->

순환

n이 짝수이면 : x^2 을 계산 -> 이 값을 $n/2$ 제곱

n이 홀수이면 : x^2 을 계산 -> 이 값을 $(n-1)/2$ 제곱 -> 여기에 x를 곱해줌

```
double slow_power(double x, int n)
{
    int i;
    double result = 1.0;
    for(i=0; i<n; i++)
        result = result * x;
    return(result);
}
```

```
power(x, n)
if n==0
    then return 1;
else if n이 짝수
    then return power(x^2, n/2);
else if n이 홀수
    then return x*power(x^2, (n-1)/2);
```

```
double power(double x, int n)
{
    if( n==0 ) return 1;
    else if ( (n%2)==0 )
        return power(x*x, n/2);
    else return x*power(x*x, (n-1)/2);
}
```

순환호출 마다 문제의 크기가 약 절반으로 줄어든다! >> $\rightarrow 2^{n-1} \rightarrow \dots \rightarrow 2^2 \rightarrow 2^1 \rightarrow 2^0$

따라서 순환으로 구현하면 시간복잡도가 $O(\log n)$ 이 된다!!

시간복잡도 : **순환** **$O(\log n)$**
 반복 **$O(n)$**

거듭제곱 성능 분석 비교 (반복 vs. 순환)

▣ 예) 2^{500} 을 1백만 번 실행

	반복적인 함수: slow_power	순환적인 함수: power
시간 복잡도	$O(n)$	$O(\log n)$
실제수행속도	7.17초	0.47초

예제 3) 피보나치 수열

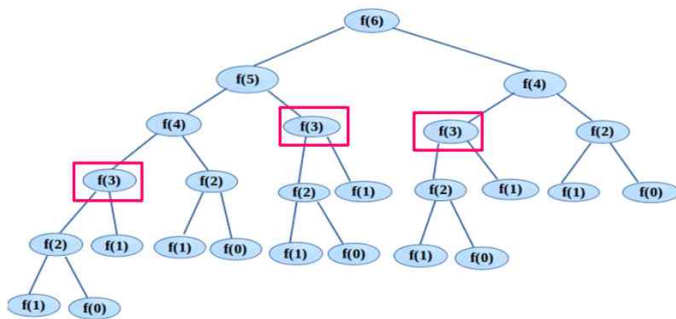
바로 앞 두 항의 합친 값이 다음 항이 됨.

★순환 호출을 사용하면 비효율적★

- : 같은 항이 중복해서 계산됨
- : n 이 커질수록 비효율성이 더욱 심해짐.

순환으로 구현한 코드

```
int fib(int n)
{
    if( n==0 ) return 0;
    if( n==1 ) return 1;
    return (fib(n-1) + fib(n-2));
}
```



반복으로 구현한 코드

```
int fib_iter(int n)
{
    if( n == 0 ) return 0;
    if( n == 1 ) return 1;

    int pp = 0;
    int p = 1;
    int result = 0;

    for( int i = 2; i <= n; i++ ) {
        result = p + pp;
        pp = p;
        p = result;
    }
    return result;
}
```

시간복잡도 : 순환 $O()$ 재귀함수를 매번 2개씩 호출함.
 반복 $O(n)$

예제 4) 하노이 탑

규칙 : 막대 A에 쌓여있는 원판 n 개를 막대 C로 옮기는 것

- * 한 번에 하나의 원판만 이동 가능
- * 맨 위에 있는 원판만 이동 가능
- * 크기가 작은 원판 위에 큰 원판이 쌓일 수 없음
- * 중간 막대를 임시적으로 이용할 수 있으나 앞의 조건들을 지켜야 함.

```
// from에 쌓여있는 n개의 원판을 tmp를 사용하여 막대 to로 이동
void hanoi_tower(int n, char from, char tmp, char to)
{
    if( n==1){
        from에서 한 개의 원판을 to로 이동
    }
    else{
        from의 맨 밑의 원판을 제외한 나머지 원판들을 tmp로 이동
        from에 있는 한 개의 원판을 to로 이동
        tmp의 원판들을 to로 이동
    }
}

hanoi_tower(n-1, from, to, tmp);
from에 있는 한 개의 원판을 to로 이동
hanoi_tower(n-1, tmp, from, to);
```

```
void hanoi_tower(int n, char from, char tmp, char to)
{
    if( n==1)
        printf("원판 1을 %c 에서 %c으로 옮긴다.\n",from,to);
    else {
        hanoi_tower(n-1, from, to, tmp);
        printf("원판 %d를 %c에서 %c으로 옮긴다.\n", n, from, to);
        hanoi_tower(n-1, tmp, from, to);
    }
}

int main(void)
{
    hanoi_tower(4, 'A', 'B', 'C');
    return 0;
}
```

Lab1 피보나치 수열 알고리즘 개선 - 동적 알고리즘(Dynamic Algorithm)

Memoization : 이미 계산된 결과를 별도의 메모리에 저장해두고 다시 사용하는 기법

```
int dynamic_fib(int i){
    int mem[1000] = {0,1, };
    if(n<2||mem[n]!=0)    //2보다 작거나 mem[]에 이미 있으면 불러다가 써라!
        return mem[n];
    else{
        mem[n]=dynamic_fib(n-1)+dynamic_fib(n-2);
        return mem[n];
    }
}
```

복잡도 : 순환(재귀) : $O(\quad)$

Memoization : $O(n)$

>> 시간복잡도가 훨씬 개선됨.

Lab2 유클리드 호제법 - 최대공약수(GCD) 구하기

- 임의의 두 자연수 a,b가 주어졌을 때 둘 중 큰 값을 a라고 가정
- a/b의 나머지를 n (즉 $n=a\%b$)
- $n==0$ 이면, b가 GCD
- 만약 $n \neq 0$ 이면
 $a \leftarrow b$
 $b \leftarrow n$
 step2부터 다시 반복

```
□ 78696 = 19332 x 4 + 1368
□ 19332 = 1368 x 14 + 180
□ 1368 = 180 x 7 + 108
□ 180 = 108 x 1 + 72
□ 108 = 72 x 1 + 36
□ 72 = 36 x 2 + 0
```

순환을 이용한 방법

```
int gcd(int a, int b) //a,b 둘 중 a가 큰값이라고 가정
{
    if(b == 0) return a;
    else return gcd(    ,    );
}
```

반복을 이용한 방법

```
int gcd(int a, int b)
{
    int n;
    while (b != 0) {
        n = a%b;
        a = b;
        b = n;
    }
    return a;
}
```

순환 코드

```
#include<stdio.h>
#include<stdlib.h>
int main(int argc, char * argv[]){
    int a = atoi(argv[1]);
    int b = atoi(argv[2]);

    int tmp;
    if(a<b){ //a에 큰 값을 할당하기 위한 조건문
        tmp = a;
        a = b;
        b = tmp;
    }

    printf("gcd : %d\n", gcd(a,b));
    return 0;
}
```

argc : main함수 실행 시 전달된 인자의 총 개수
argv : argv[0]은 파일이름, argv[1]부터 사용자 입력.

atoi() : ASCII to Int

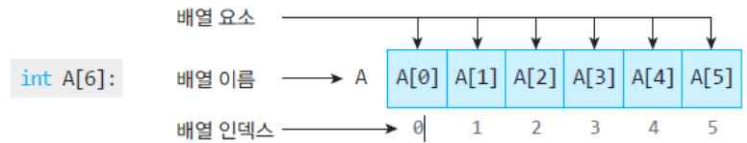
```
int gcd(int a, int b) {
    if (b == 0)
        return a;    // 종료 조건: b가 0이면 a가 GCD
    else
        return gcd(b, a % b); // 재귀 호출
}
```

3장 배열, 구조체, 포인터

배열

같은 형의 변수를 여러 개 만드는 경우에 사용
인덱스를 주면 해당하는 값이 대응되는 구조

ex) A1, A2, A3, A4, A5, A6 -> int A[6]



데이터 타입 : <인덱스, 값> 쌍의 집합

연산 get 연산 : 배열과 인덱스가 주어지면 그 위치의 값 반환
 set 연산 : 배열, 인덱스, 값을 받아서 새로운 인덱스 위치에 값을 저장

1차원 배열

index는 0부터 시작

배열이름 == 배열의 첫 번째 원소의 주소. 배열의 시작을 가리키는 포인터

배열은 메모리의 연속된 위치에 저장됨.

A[i]를 불러올 때는 base+i*sizeof(int)로 가져옴.

```
int A[6]= {1,2,3,4,5,6};
```

```
int A[] = {1,2,3,4,5,6};                      << 크기를 생략하면 초기값을 나열한 수만큼 메모리 할당됨.
```

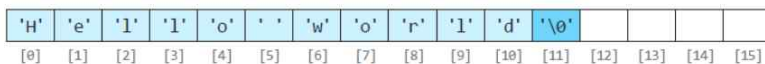
2차원 배열

1차원 배열처럼 크기를 생략하고 초기값을 저장할 수 있지만 첫 번째 크기(2차원 행렬에서는 행)만 생략가능

문자열

문자열의 끝을 나타내는 **wo** 또는 **NULL** 이 마지막에 추가됨. >> 실제 문자열보다 1바이트 더 필요.

```
char s1[16] = "Hello World";
```



□ 선언 및 초기화: 차이?

- s2[16] = {'H','e','l','l','o',' ','W','o','r','l','d','\0'};
- s3[] = "Hello World";
- s4[16] = {'H','e','l','l','o',' ','W','o','r','l','d'};

하나씩 초기화 함.

자동으로 크기가 12로 결정됨.

끝에 NULL문자를 넣지 않음! > 문자열이 아님.

구조체

: 타입이 다른 데이터 모임

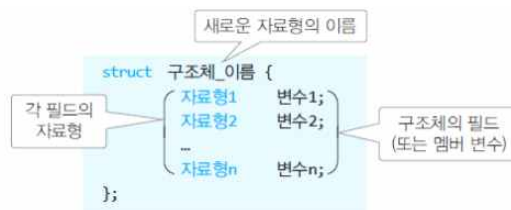
<-> 배열 : 타입이 같은 데이터 모임

구조체는 사용하기 전에 반드시 **정의**를 해야함.

typedef로 만들기 가능.

typedef struct뒤에 생략 가능.
(단 자기참조 시 불가능)

```
struct Student {  
    int id;  
    char name[100];  
    double score;  
};  
  
struct Student a;
```



```
typedef struct Student_t {  
    int id;                      // 학생의 학번  
    char name[20];              // 학생의 이름  
    double score;               // 키를 나타내는 실수값  
} Student;  
  
Student a;
```

이 경우 Student는 새로운 데이터 타입의 이름이 됨

선언과 동시에 초기화 하는 방법

Student a = {123456, "홍길동", 97.3}

구조체 멤버에 접근하기

```
a.id = 123456;
strcpy(a.name, "홍길동");           //a.name = "홍길동";은 오류발생
a.score = 97.3;
```

구조체의 연산 : 구조체는 기본적으로 대입연산자(=)만 지원

a = b

구조체를 포함하는 구조체, 구조체로 배열을 선언하는 것도 가능!

구조체를 포함 하는 구조체

```
typedef struct {
    int month;
    int date;
} Birthday;

typedef struct {
    char name[20];
    Birthday birthday;
} Friend ;
```

구조체 배열

```
void main() {
    Friend list[45];           // 구조체의 배열 선언
    list[0].birthday.month = 7;
    list[0].birthday.date = 5;
    strcpy_s(list[0].name, 20, "Minyoung");
}
```

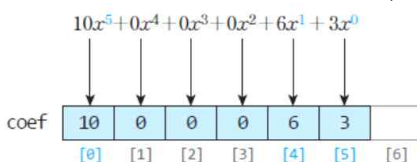
strcpy_s는 vs에서 사용. 20자 이내로 cpy하겠다

다항식의 표현

다항식의 일반적인 형태 : $p(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$

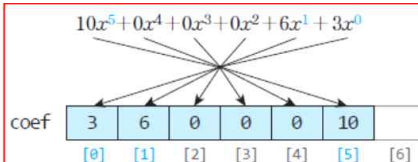
```
#define MAX_DEGREE 1000
typedef struct {
    int degree;           //차수
    float coef[MAX_DEGREE]; //계수
} Polynomial ;
```

계수를 최고차항부터 저장하거나, 반대로 최고차항을 맨 뒤에 저장하는 방법이 있음.



(a) 방법 1: 최고차항의 계수부터
순서대로 저장하는 방법

Polynomial a = { 5, { 10, 0, 0, 0, 6, 3 } };



(b) 방법 2: 최고차항의 계수를
맨 뒤에 저장하는 방법

Polynomial a = { 5, { 3, 6, 0, 0, 0, 10 } };

=> 코드 참고! 3-17, 18

희소 다항식

: 차수가 매우 크고 0이 아닌 계수항이 몇 개 없는 다항식(예 : $0x^{000} + 6$)

>> 모든 계수를 저장하면 비효율적이다!

```
typedef struct {
    int     expon; //지수
    float    coeff; //계수
} Term;
```

```
typedef struct {
    int nTerms; //항의 개수
    Term term[MAX_TERMS];
} SparsePoly;
```

>> 구조체 안의 구조체!

희소 행렬

- : 많은 항들이 0으로 구성되어 있는 행렬
- >> 2차원 배열로 구현하면 메모리 낭비가 심함!!
- >> 0이 아닌 항만 1차원 배열로 모아서 행렬을 표현.

$$M = \begin{bmatrix} 0 & 0 & 2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 7 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 3 \\ 0 & 5 & 0 & 0 & 0 & 0 \end{bmatrix}$$

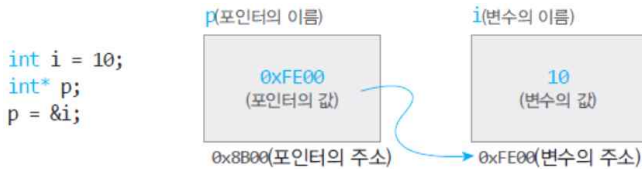
희소행렬은 전치행렬(Transpose)로 바꾸기 매우 간편.

- >> 값은 그대로 두고 행과 열만 switch

	행번호	열번호	값	
[0]	0	2	2	행열 요소 (행, 열, 값)
[1]	2	3	7	
[2]	3	0	1	
[3]	4	5	3	
[4]	5	1	5	

포인터

: 메모리의 주소를 저장하는 변수. 다른 변수의 주소를 저장하는 데 사용



&연산자 : 변수의 주소를 반환

* 연산자 : 포인터가 가리키는 곳의 내용을 반환(추출) - Dereferencing

□ 'p = '

- 일반 변수처럼 포인터 변수가 저장된 메모리 공간
- 포인터 변수의 값 (즉, 메모리 주소)이 변경됨

```
int *p;
p = (int *)0x0000006C; //ptr 변수에 맞는 자료형으로 타입변환 후 대입
```

□ '*p = '

- 포인터가 가리키는 메모리 공간
- 포인터가 가리키는 대상의 값이 변경됨

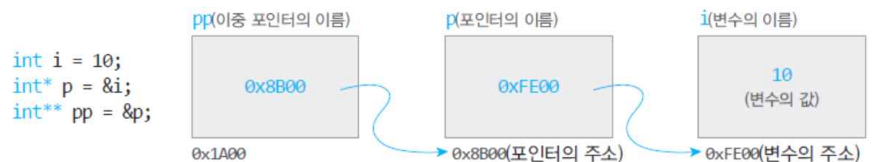
```
int *p;
p = (int *)0x0000006C;
*p = 1234; //포인터가 가리키는 대상에 값을 대입
```

□ 주의: 실제 프로그램에서는 메모리 주소를 직접 넣는 경우는 거의 없음

이중포인터 : 포인터를 가리키는 포인터!

>> 포인터의 배열이다...?

>> 2차원 데이터를 다룰 때 자주 사용



배열, 구조체와 포인터

배열 이름은 배열의 시작을 가리키는 포인터이다!

```

int A[5], *p=NULL; // 배열 A와 포인터 p 선언
p = A;             // p에 배열의 주소(배열 이름)이 복사됨
p[3] = 20;         // 배열처럼 사용할 수 있음(p[3]은 A[3]과 동일)

```

구조체의 포인터

-> 기호를 통해서 역참조 가능!

```

typedef struct { // 다항식 구조체 선언
    int degree;
    float coef[MAX_DEGREE];
} Polynomial;

Polynomial a; // 다항식 구조체 변수 a 선언
Polynomial* p=NULL; // 다항식 포인터 p 선언
p = &a; // 이제 p가 구조체 a를 가리킴
a.degree = 5; // 구조체 a를 통한 멤버 접근: . 연산자 사용
p->coef[0] = 1; // 포인터 p를 통한 멤버 접근: -> 연산자 사용

```

자기참조 구조체(Self-referential structure)

: 구조체를 정의할 때 멤버중에 자기 자신을 가리키는 포인터가 존재하는 구조체

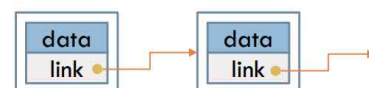
```

typedef struct ListNode {
    char data[10];
    struct ListNode* link;
} Node;

```

<< 여기서 ListNode를 생략할 수 없음!!!!

생략하면 여기서 정의 전이기 때문에 사용 불가능....



동적 메모리 할당

필요할 때 필요한 만큼 메모리를 할당 받아 사용하고, 사용이 끝나면 반납하는 기능 > 효율적으로 사용 가능!

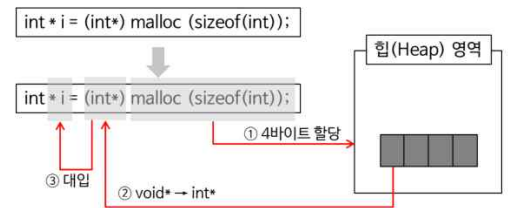
`void * malloc(size_t size)`

`size_t`는 unsigned int

바이트(byte)단위로 메모리를 할당하고 시작 포인터를 반환

`void`형 포인터를 리턴해서 타입캐스팅을 해 줘야 함!

할당 실패 시 `NULL`포인터 반환



`void * calloc(size_t count, size_t size)`

Size크기의 변수를 `count`개수만큼 저장할 수 있는 메모리 공간 할당

`malloc()`과 달리 할당된 공간을 모두 0으로 초기화한다! -> 따로 초기화를 안 해줘도 됨.

```
int* a = (int*)malloc(5 * sizeof(int)); // 초기화 안 됨
```

```
int* b = (int*)calloc(5, sizeof(int)); // 0으로 초기화
```

`free()` - 동적 메모리 해제 함수

할당되었던 메모리 블록을 반드시 시스템에 반납해야 함.

`void * realloc(void *ptr, size_t size)`

`ptr`가 가리키는 메모리 주소크기를 `size` 바이트로 확장 또는 축소

```
int* data = (int*)malloc(sizeof(int)*100); // int 100개 크기의 메모리 할당
data[99] = 30; // data[0]~data[99] 사용 가능
data = (int*)realloc(data, sizeof(int)*200); // int 200개 크기로 확장
data[199] = 60; // 이제 data[0]~data[199] 사용 가능
free(data); // 반납 방법은 동일
```

Lab 1: 함수의 매개변수로 포인터 사용하기

```
#include <stdio.h>
```

```
void swap(int x, int y){
```

```
    int tmp;
```

```
    tmp = x;
```

```
    x=y;
```

```
    t=tmp;
```

```
}
```

```
int main(void){
```

```
    int a=1, b=2;
```

```
    printf("전 : a=%d, b=%d\n", a,b);
```

```
    swap(a,b);
```

```
    printf("후 : a=%d, b=%d\n", a,b);
```

```
    return 0;
```

```
}
```

```
>>> 출력 결과 전 a=1, b=2
```

```
후 a=1, b=2
```

```
#include <stdio.h>
```

```
void swap(int *x, int *y){
```

```
    int tmp;
```

```
    tmp = *x;
```

```
    *x = *y;
```

```
    *t=tmp;
```

```
}
```

```
int main(void){
```

```
    int a=1, b=2;
```

```
    printf("전 : a=%d, b=%d\n", a,b);
```

```
    swap(&a,&b);
```

```
    printf("후 : a=%d, b=%d\n", a,b);
```

```
    return 0;
```

```
전 a=1, b=2
```

```
후 a=2, b=1
```

call by value : 함수가 인수로 전달받은 값을 복사하여 처리하는 방식. 원본 값 변경되지 않음

call by reference : 함수 호출시 인수로 변수의 참조 값(주소 값)을 함수 내부로 전달하는 방식. 원본 값 변경됨

Lab 2: 함수의 매개변수로 포인터 사용하기

배열의 이름은 포인터?

```
int main(){
    char arr[100] = "hello";
    char *ptr = "world";
    arr=ptr;    //불가능!
}
```

```
int main(){
    char arr[100] = "hello";
    char *ptr = "world";
    ptr=arr;    //가능.
}
```

★배열의 이름은 포인터가 아니다!★ 배열의 시작 위치를 가지고 있는 const이다!

```
char arr[10]="hello1";
char *ptr = "hello2";
ptr[0]='H'; //불가능! Read-Only
arr[0]='H';
```

```
char arr[10]="hello1";
char *ptr =arr;
ptr[0]='H';
arr[0]='H';
```

배열과 포인터의 차이

문자열을 초기화할 때 배열의 원소를 변경할 수 있지만, 포인터는 배열의 원소 변경 불가!
포인터로 초기화할 경우 상수적인 성격이 된다! 읽기전용 Read-Only

strcat(dst, src) //src를 dst에 붙인다! 앞에있는 dst가 기준

```
char arr[20] = "hello, ";
char *ptr = "world";
strcat(arr,ptr);    //가능
```

```
char arr[20]="hello, ";
char *p =arr;
char *ptr = "world";
strcat (p,ptr);
```

```
char *ptr = "hello";
char arr[20] = "world";
strcat(ptr, arr);    //불가능!
```

```
char *ptr = "hello, ";
strcat(ptr, "world");    //불가능!
```

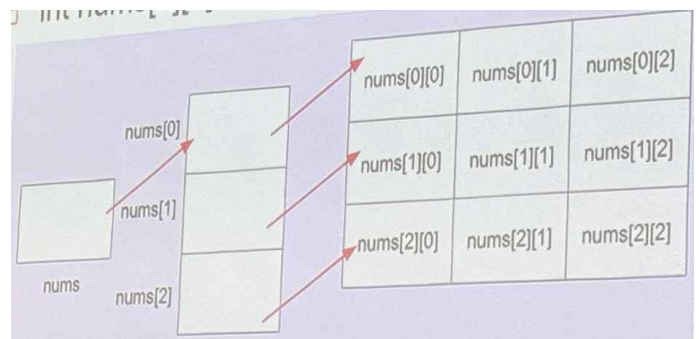
★포인터 변수에는 strcat 불가능!

Lab 3: 이중 포인터 예제

2차원 배열은 이중 포인터 개념과 비슷함.

```
int nums[3][3];
nums[0] == *(nums+0)
nums[1] == *(nums+1)
nums[2] == *(nums+2)

nums[0][1] == *(nums[0]+1) == *(* (nums+0)+1)
nums[2][1] == *(nums[2]+1) == *(* (nums+2)+1)
```



Lab 4: 이중 포인터 예제

main()함수의 argv 매개 변수

```
int main(int argc, char * argv[])
```

argc : argument count 인자의 개수

argv : argument vector 인자 리스트

Main 함수에 어떻게 인자를 전달할 수 있을까?

프로그램이 실행되기 전에 인자를 전달해야 함.

(실행파일명.exe) (인자) (인자)

ex) test.exe hello CAU 123

argc : 4

argv * argv[] = {"test.exe", "hello", "CAU", "123"}

주의 : 숫자도 문자로 인식함!

char * argv[]는 인자로 전달한 내용이 저장되는 문자열 배열.

argv[]는 포인터배열이다!

4장 스택

스택(stack)

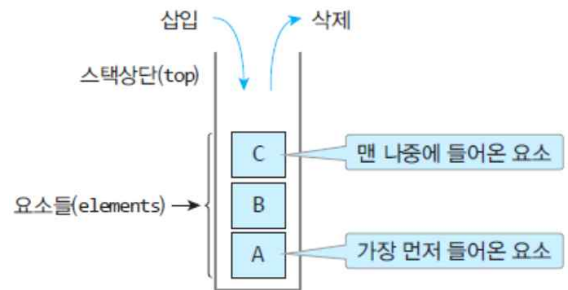
: 쌓아놓은 더미. 후입선출(LIFO: Last In First Out)

가장 최근에 들어온 데이터가 가장 먼저 나감

스택 상단(top), 하단(bottom)

요소(element) : 스택에 저장되는 것

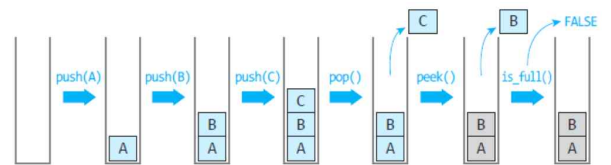
요소의 입출력 : 스택 상단에서만 일어남.



스택 추상 자료형

연산

- **init()**: 스택을 초기화한다. //top 포인터를 -1로 한다
- **push(e)**: 요소 e를 스택의 맨 위에 추가한다.
- **pop()**: 스택의 맨 위에 있는 요소를 꺼내 반환한다.
- **is_empty()**: 스택이 비었으면 TRUE를 아니면 FALSE를 반환한다.
- **is_full()**: 스택이 가득 차 있으면 TRUE를 아니면 FALSE를 반환한다.
- **peek()**: 스택의 맨 위에 있는 항목을 삭제하지 않고 반환한다.



pop()은 맨 위엿걸 꺼내는 것이고, peek()은 맨 위엿걸 리턴만 한다. 삭제X.

오버플로우(Overflow) : stack이 꽉 찼을 때 무언가 push 하려고 하면 overflow

언더플로우(Underflow) : 아무것도 없는데 pop이나 peek을 하려고 하면 underflow

>> is_empty()나 is_full()연산을 이용해 스택 상태를 미리 검사해야 한다.

스택 활용

시스템 스택을 이용한 함수호출
 편집기의 '되돌리기' 기능
 웹 브라우저의 '이전 페이지로 이동' 기능

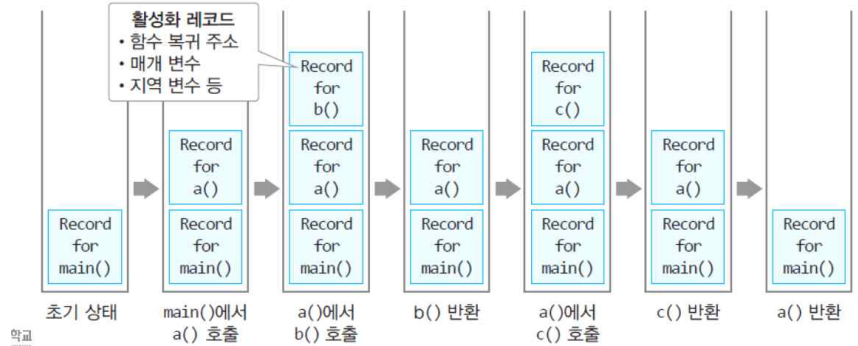
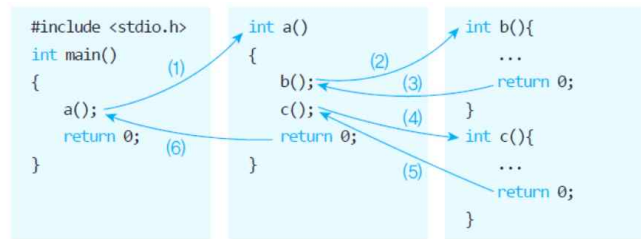
- * 소스 코드의 괄호 사용 검사
- * 계산기 프로그램(수식 계산)
- * 함수 호출과 반환을 위한 시스템 스택

구현 방법 : List 구조, Linked-list 구조

스택의 구현

1차원 배열 : `data[MAX_SIZE]`
 MAX_SIZE : 스택의 최대 저장 용량

가장 최근 입력된 요소의 위치를 나타내는 변수 : **top**
 스택이 empty면 **top = -1**
data[0] : 가장 먼저 들어온 요소
data[top] : 가장 최근 들어온 요소



is_empty() 비어있는가?
 top == -1인가?

is_full() 꽉 찼는가?
 top == capacity-1인가?

push() 새로 넣기

pop() 상단요소를 삭제하고 출력

peek() 상단요소를 삭제하지 않고 출력

알고리즘 3.1 스택의 공백 상태 검사

```
01 is_empty()
02     if top == -1 : return TRUE
03     else : return FALSE
```

알고리즘 3.2 스택의 포화 상태 검사

```
01 is_full()
02     if top == MAX_SIZE-1 : return TRUE
03     else : return FALSE
```

알고리즘 3.4 스택의 삽입 연산

```
01 push(e)
02     if is_full() :
03         error "overflow"
04     else :
05         top ← top + 1
06         data[top] ← e
```

알고리즘 3.5 스택의 삭제 연산

```
01 pop()
02     if is_empty() :
03         error "underflow"
04     else :
05         top ← top-1
06         return data[top+1]
```

알고리즘 3.6 스택의 참조(peek) 연산

```
01 peek()
02     if is_empty() :
03         error "underflow"
04     else : return data[top]
```

```
void push(element item){
    if (is_full()){ fprintf(stderr,"SO Error\n"); return ;
    }
    else stack[++top] = item;
```

```
element pop(){
    if (is_empty()){ fprintf(stderr,"SO Error\n"); return ;
    }
    else return stack[top--];
```

```
element peek(){
    if (is_empty()){ fprintf(stderr,"SO Error\n"); return ;
    }
    else return stack[top];
```

스택에 저장되어야 하는 요소가 정수나 문자가 아니고 더 복잡한 구조를 갖는 요소이면?
 -> 스택을 구조체 배열로 구현해야 함.

스택의 응용 : 괄호검사

괄호의 종류 : [] { } ()

괄호 검사 조건

1. 왼쪽 괄호의 개수와 오른쪽 괄호의 개수가 같아야 한다.
2. 같은 괄호에서 왼쪽 괄호는 오른쪽 괄호보다 먼저 나와야 한다.
3. 서로 다른 종류의 괄호가 교차하여 쌍을 이루면 안 된다.

```

01 check_matching(expr)
02     init_stack()
03     while (입력 expr의 끝이 아니면) :
04         ch ← expr의 다음 글자
05         if ch ∈ { '(', '[', '{' } :
06             push(ch)
07         else if ch ∈ { ')', ']', '}' } :
08             if is_empty() : 오류(조건 2)
09             else :
10                 open_ch ← pop()
11                 if (ch 와 open_ch가 같은 짝이 아니면) :
12                     오류(조건 3)
13
14     if is_empty() = FALSE :
15         오류(조건 1)
16     else : 검사 성공
    
```

스택의 응용 : 괄호검사

수식 표기법 : prefix, infix, postfix

전위(prefix)	중위(infix)	후위(postfix)
연산자 피연산자1 피연산자2	피연산자1 연산자 피연산자2	피연산자1 피연산자2 연산자
+ A B	A + B	A B +
+ 5 * A B	5 + A * B	5 A B * +

Postfix의 계산방식

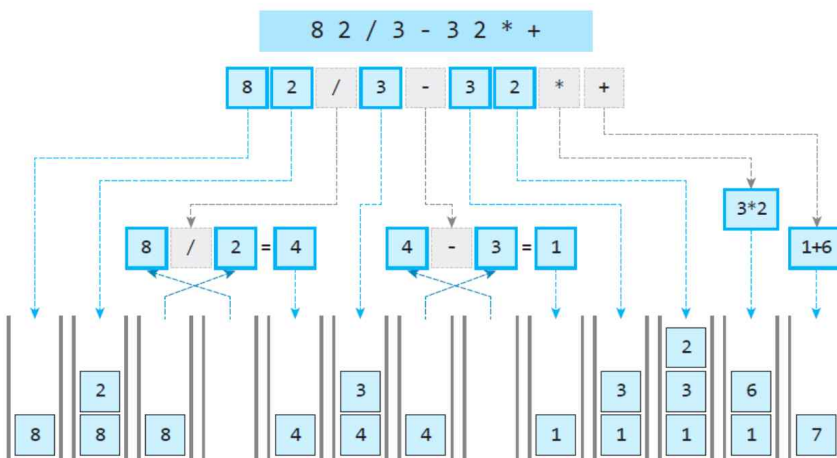
숫자를 스캔.

피연산자이면 스택에 push. 연산자이면 필요한 수만큼 피연산자를 pop

연산을 하고 다시 그 결과를 push

Postfix 계산 예: 82/3-32*+

코드 확인! 4-30장



```

01 eval_postfix(expr)
02     init_stack()
03     for elem in expr :
04         if (elem이 피연산자이면) :
05             push(elem)
06         else if (elem이 연산자 op이면) :
07             second ← pop()
08             first ← pop()
09             temp ← first op second
10             push(temp)
11     return pop()
    
```

Infix -> Postfix 변환

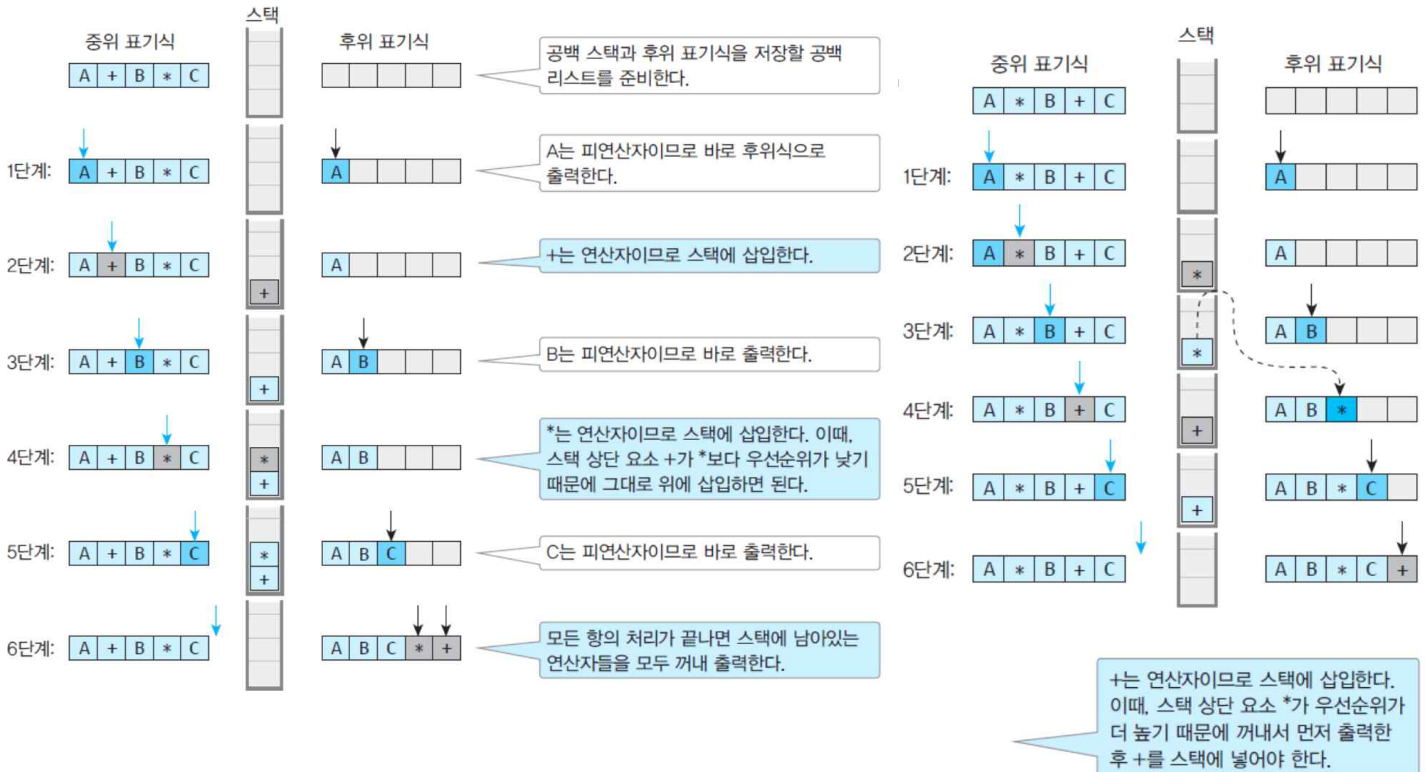
순서대로 스캔한다.

피연산자를 만나면 곧바로 출력한다.

연산자를 만나면 스택에 저장한다.

이때 연산자의 우선순위를 고려하여 연산자를 저장 또는 출력
모든 항의 처리가 끝나면 스택에 남아 있는 연산자들을 모두 꺼내 출력한다.

연산자 우선순위 규칙 : 스택에 넣을 연산자보다 우선 순위가 높은 연산자는 모두 먼저 출력한 후 현재 연산자를 스택에 넣는다. 우선순위가 같은 연산자도 먼저 출력해야 한다.



괄호가 포함될 경우

왼쪽 괄호는 무조건 스택에 넣는다 - 제일 우선순위가 낮은 연산자로 취급한다.

오른쪽 괄호를 만나면 하나의 왼쪽 괄호가 pop될 때 **괄호 위에 쌓여 있는 모든 연산자를 출력한다.**

```

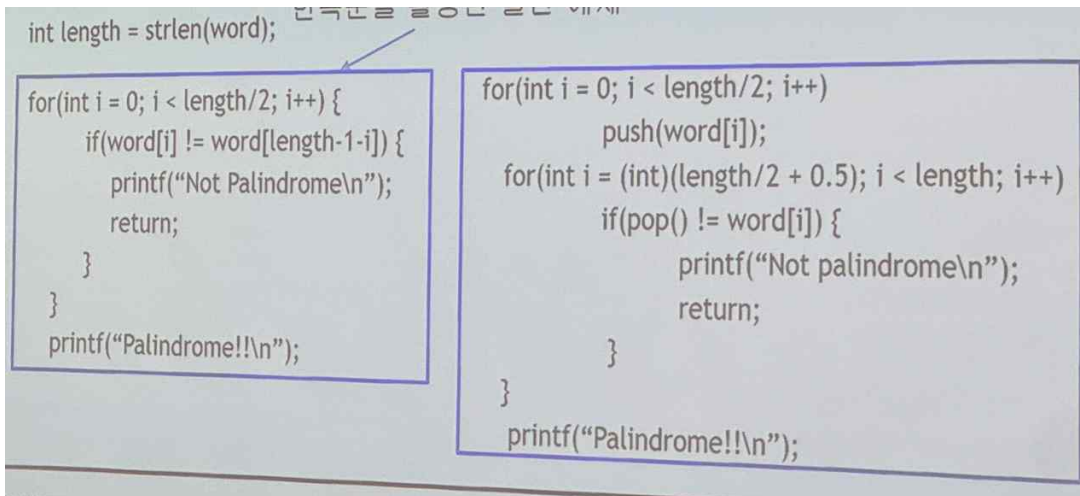
01 infix_to_postfix(expr)
02   init_stack()
03   while (expr에 처리할 항이 남아 있으면) :
04       term ← 다음에 처리할 항
05       if (term이 피연산자이면) :
06           term을 출력      // 후위
07       else if (term이 왼쪽 괄호이면) :
08           push(term)
09       else if (term이 오른쪽 괄호이면) :
10           e ← pop()
11           while (e ≠ 왼쪽괄호) :
12               e를 출력
13               e ← pop()
14       else if (term이 연산자) :
15           while (peek()의 우선순위 ≥ term의 우선순위) :
16               e ← pop()
17               e를 출력
18           push(term)
19
20   while is_empty() = FALSE :
21       e ← pop()
22       e를 출력
    
```

Lab : Palindrome(회문)

앞뒤 어느 쪽에서 읽어도 같은 단어 또는 문장

ex) level, race car, Madam

구두점, 스페이스, 대소문자 무시



5장 큐

큐(queue)

: 먼저 들어온 데이터가 먼저 나가는 자료구조

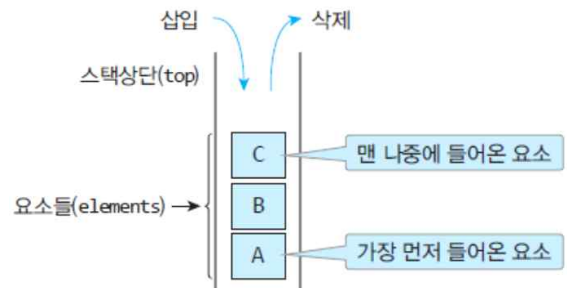
선입선출 FIFO First-In First-Out

구조

전단(front), 후단(rear)

요소(element) : 큐에 저장되는 것.

요소의 입출력 : 출력(front), 입력(rear)



연산 데이터: 선입선출(FIFO)의 접근 방법을 유지하는 요소들의 모음

연산

- `init()`: 큐를 초기화한다.
- `enqueue(e)`: 요소 `e`를 큐의 맨 뒤에 삽입한다.
- `dequeue()`: 큐의 맨 앞 요소를 삭제하고 반환한다.
- `is_empty()`: 큐가 비어있으면 `TRUE`를 아니면 `FALSE`를 반환한다.
- `is_full()`: 큐가 가득 차 있으면 `TRUE`를 아니면 `FALSE`를 반환한다.
- `peek()`: 큐의 맨 앞 요소를 삭제하지 않고 반환한다.

활용

버퍼(buffer) : 갑자기 데이터가 몰려드는 경우 이들을 잠시 보관하는 장소

컴퓨터 시뮬레이션 : 비행기 이착륙, 인터넷 데이터 패킷 전송 등

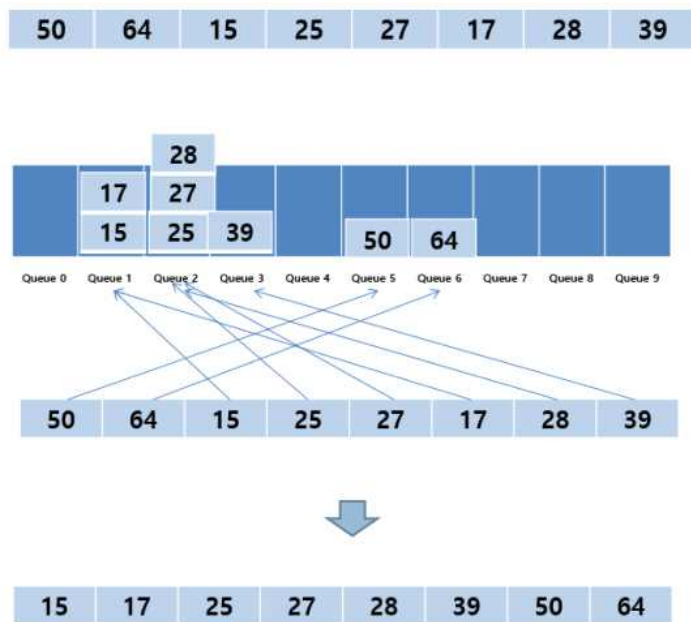
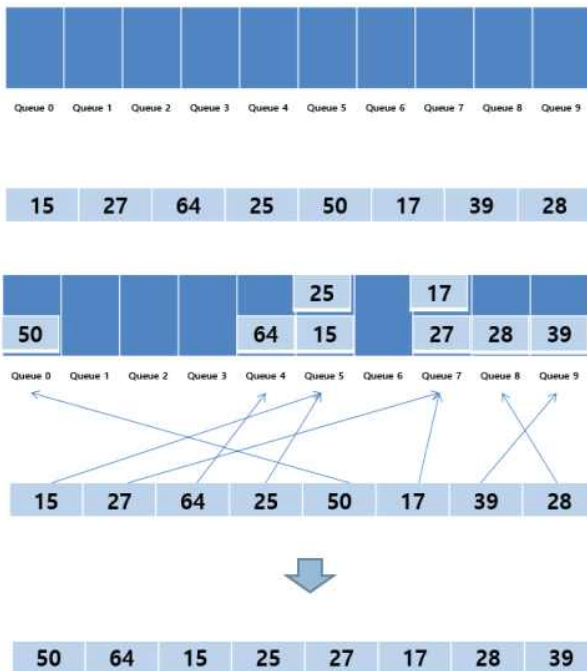
다양한 알고리즘 : 레벨 순회, 너비우선탐색, 기수정렬(Radix sort) 등

Radix Sort(기수 정렬)

자릿수 만큼의 len길이의 큐 리스트가 필요하다. 2자리수이면 2개

장점 : 빠르다.

단점 : 자릿수가 정해져 있어야 계산하기가 가능.

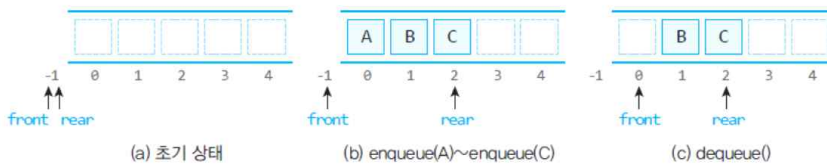


시간복잡도 : $O(dn)$ - d는 digit 자릿수.
공간복잡도 : $O(n+d)$

선형 큐

선형 큐의 동작

□ 선형 큐의 동작



- 큐의 초기화
 - front 와 rear 모두 -1
- Enqueue 마다 rear ++
- Dequeue 마다 front ++
- is_full(): **rear == MAX_SIZE - 1**
- is_empty(): **rear == front**

코드 보기!
큐 9~12페이지

선형 큐의 응용 - OS 작업 스케줄링

선형 큐의 문제점 : front와 rear 인덱스가 증가하기만 하므로 언젠가 배열 끝에 도달함.
배열 끝에 도달시 enqueue하려면 이전 요소들을 모두 앞으로 옮긴 후 데이터 삽입 가능
>> 비효율적이다!

원형 큐


```

#define MAX_DEGREE 1000
typedef struct {
    int degree;
    float coef[MAX_DEGREE];
} Polynomial ;

```

```

26 Polynomial add(Polynomial a, Polynomial b) // 다항식 a와
27 {
28     Polynomial p;
29     p.degree = (a.degree > b.degree) ? a.degree : b.degree;
30
31     for (int i=0; i<=p.degree; i++)
32         p.coef[i] = ((i <= a.degree) ? a.coef[i] : 0)
33                 + ((i <= b.degree) ? b.coef[i] : 0);
34     return p;
35 }
36
37 void print_poly(Polynomial p, char str[]) // 다항식을 화면에 보기 좋게 출력
38 {
39     printf(" %s", str);
40     for (int i = p.degree; i > 0; i--)
41         printf("%5.1f x^%d + ", p.coef[i], i);
42     printf("%4.1f\n", p.coef[0]);
43 }

```

```

44 void main()
45 {
46     Polynomial a = { 5, { 3, 6, 0, 0, 0, 10} };
47     Polynomial b = { 4, { 7, 0, 5, 0, 1} };
48     Polynomial c = add(a, b);
49     print_poly(a, " A = ");
50     print_poly(b, " B = ");
51     print_poly(c, "A+B= ");
52     printf("A(1)= %f\n", evaluate(a, 1.0f));
53     printf("B(2)= %f\n", evaluate(b, 2.0f));
54 }

```

i가 a의 차수 이하이면, 그 항의 계수를 더함.
 b도 마찬가지로
 그렇지 않으면 0을 더함

```

typedef struct {
    int     expon;
    float   coeff;
} Term;

```

```

typedef struct {
    int nTerms;
    Term term[MAX_TERMS];
} SparsePoly;

```

```

21 SparsePoly add(SparsePoly a, SparsePoly b) { // 다항식 a와 b의 합
22     SparsePoly c;
23     c.nTerms = 0;
24     for (int i=0, j=0 ; i < a.nTerms || j < b.nTerms;) {
25         if (i == a.nTerms || a.term[i].expo < b.term[j].expo)
26             c.term[c.nTerms++] = b.term[j++];
27         else if (j == b.nTerms || a.term[i].expo > b.term[j].expo)
28             c.term[c.nTerms++] = a.term[i++];
29         else {
30             c.term[c.nTerms] = a.term[i++];
31             c.term[c.nTerms].coef += b.term[j++].coef;
32         }
33     }
34     return c;
35 }

```

```

47 void main()
48 {
49     SparsePoly a = { 2, {{100, 10.0f}, {0, 6.0f}} };
50     SparsePoly b = { 2, {{50, 1.1f}, {3, 2.0f}} };
51     SparsePoly c = add(a, b);
52     print_spoly(a, " A = ");
53     print_spoly(b, " B = ");
54     print_spoly(c, "A+B= ");
55     printf("A(1)= %f\n", evaluate(a, 1.0f));
56     printf("B(2)= %f\n", evaluate(b, 2.0f));
57 }

```



```

01 #include <stdio.h>
02 typedef struct {
03     int row;    // 행 번호
04     int col;    // 열 번호
05     int value;  // 행렬 요소의 값
06 } Elem;        // 하나의 행렬 요소

15 void transpose_mat2(Elem m[], int len)
16 {
17     for (int i = 0; i < len; i++) {
18         int tmp = m[i].row;
19         m[i].row = m[i].col;
20         m[i].col = tmp;
21     }
22 }

```

행렬의 각 항을 위한 구조체

전치 연산을 위해 각 항의 행 번호와 열 번호를 교환
항의 값은 변경되지 않음

```

Elem mat[5] = { {0,3,11}, {2,5,42}, {3,1,33}, {4,4,87}, {5,4,55} };
transpose_mat2(mat, 5);

```

```

#include <stdio.h>
#include <stdlib.h>

```

```

#define MAX_STACK_SIZE 100    // 스택의 최대 크기
typedef int element;          // 데이터의 자료형
element stack[MAX_STACK_SIZE]; // 1차원 배열
int top = -1;

```

스택이 전역 변수로 구현됨

```

// 공백 상태 검출 함수
int is_empty()
{

```

```

    if (top == -1) return 1;
    else return 0;
}

```

이 문장은 다음과 같음
return (top == -1);

```

// 포화 상태 검출 함수
int is_full()
{
    return (top == (MAX_STACK_SIZE - 1));
}

```

```

// 삽입 함수
void push(element item)
{
    if (is_full()) {
        fprintf(stderr, "Stack Overflow Error!!\n");
        return;
    }
    else stack[++top] = item;
}

// 삭제 함수
element pop()
{
    if (is_empty()) {
        fprintf(stderr, "Stack Empty Error!!\n");
        exit(1);
    }
    else return stack[top--];
}

```

```

element peek()
{
    if (is_empty()) {
        fprintf(stderr, "Stack Empty Error!!\n");
        exit(1);
    }
    else return stack[top];
}

```

```

int main(void)
{
    element input = {123456, "Park", "Seoul"};
    element output;

    push(input);
    output = pop();

    printf("학번: %d\n", output.student_no);
    printf("이름: %s\n", output.name);
    printf("주소: %s\n", output.address);
    return 0;
}

```

```

#include <stdio.h>
#include <stdlib.h>
#define MAX_SIZE 100
typedef char Element; // 스택 요소 Element의 자료형
#include "ArrayStack.h" // 스택의 ADT(데이터와 연산) 포함
int check_matching(char expr[])
{
    int i = 0, prev;
    init_stack();
    while (expr[i] != '\0') {
        char ch = expr[i++];
        if (ch == '[' || ch == '(' || ch == '{') push(ch);
        else if (ch == ']' || ch == ')' || ch == '}') {
            if (is_empty()) return 2;
            prev = pop();
            if ((ch == ']' && prev != '[') || (ch == ')' && prev != '(') || (ch == '}' && prev != '{'))
                return 3;
        }
        // Pop한 괄호와 짝이 맞지 않으면 조건 3위반
    }
    if (!is_empty()) return 1;
    else return 0; // 괄호 정상
}
// 모두 처리 후 스택이 공백이 아니면 조건 1위반

void main()
{
    char expr[4][80] = {
        "{A[(i+1)]=0;}",
        "if((i==0) && (j==0)",
        "while(n<8)){n++;}",
        "arr[(i+1)] = 0;"
    };

    for (int i = 0; i < 4; i++) {
        int errCode = check_matching(expr[i]);
        if (errCode == 0) printf("%-20s -> 정상\n", expr[i]);
        else printf("%-20s -> 오류(조건%d 위반)\n", expr[i], errCode);
    }
}

```

$A + B$
 $A + B * C$
 $(A + B) * C$
 $A * B + C * D$
 $A + B + C$
 $A + (B - C) * D$
 $(A + B) * (C + D)$
 $(A + B * (C - D)) / E$
 $A + ((B + C) * (D + E))$
 $A * (B + C * (D + E))$
 $A / B - C + D * E$

$A B +$
 $A B C * +$
 $A B + C *$
 $A B * C D * +$
 $A B + C +$
 $A B C - D * +$
 $A B + C D + *$
 $A B C D - * + E /$
 $A B C + D E + * +$
 $A B C D E + * + *$
 $A B / C - D E * +$

같아도 pop

```

#include <stdio.h>
#include <stdlib.h>
#define MAX_SIZE 100
typedef char Element; // 스택 요소 Element의 자료형
#include "ArrayStack.h" // 스택의 ADT(데이터와 연산) 포함

```

```
int priority(char op)
```

```

{
    switch (op) {
        case '(': case ')': return 0;
        case '+': case '-': return 1;
        case '*': case '/': return 2;
    }
    return -1;
}

```

```
07 double eval_postfix(char expr[])
```

```
12 while (expr[i] != '\0') {
```

```
13     char c = expr[i++];
```

```

14     if (c >= '0' && c <= '9') {
15         double num = (double)c - '0';
16         push(num);
17     }

```

문자가 피연산자(0~9로 제한)이면 숫자로 변환해 스택에 삽입

```

18     else if (c=='+' || c=='-' || c=='*' || c=='/') {
19         double val2 = pop(); // 두 번째 피연산자
20         double val1 = pop(); // 첫 번째 피연산자
21         switch (c) {
22             case '+': push(val1 + val2); break;
23             case '-': push(val1 - val2); break;
24             case '*': push(val1 * val2); break;
25             case '/': push(val1 / val2); break;
26         }
27     }

```

문자가 연산자(사칙 연산 + - * /로 제한)이면 스택에서 두 개의 피연산자를 꺼내 계산한 후, 결과를 다시 스택에 저장

```

28 }
29 return pop();

```

```
void infix_to_postfix(char expr[])
```

```
{
    init_stack();
    int i = 0;
    while (expr[i] != '\0') {
        char c = expr[i++];
        if ((c >= '0' && c <= '9')) printf("%c ", c);
        else if (c == '(') push(c);
        else if (c == ')') {
            while (is_empty() == 0) {
                char op = pop();
                if (op == '(') break;
                else printf("%c ", op);
            }
        }
        else if (c == '+' || c == '-' || c == '*' || c == '/') {
            while (is_empty() == 0) {
                char op = peek();
                if (priority(c) <= priority(op)) {
                    printf("%c ", op);
                    pop();
                } else break;
            }
            push(c);
        }
    }
    while (is_empty() == 0)
        printf("%c ", pop());
    printf("\n");
}
```

← 피연산자는 바로 출력

← 열린 괄호면 스택에 삽입

← 닫힌 괄호면 왼쪽 괄호가 나올때까지
모든 연산자를 꺼내 출력

← 연산자면 스택에서 현재 연산자
보다 우선순위가 같거나 높은
모든 연산자를 꺼내 출력.
마지막으로 현재 연산자를 삽입

← 모든 항의 모든 항의 처리가 끝나면 스택에 남아
있는 연산자들을 모두 꺼내 출력한다

```
Void main()
```

```
{
    char expr[2][80] = { "8 / 2 - 3 + (3 * 2)", "1 / 2 * 4 * (1 / 4)" };

    printf("infix: %s ==> postfix:", expr[0]);
    infix_to_postfix(expr[0]);
    printf("infix: %s ==> postfix:", expr[1]);
    infix_to_postfix(expr[1]);
}
```

QUEUE

```
#include <stdio.h>
#include <stdlib.h>
#define MAX_QUEUE_SIZE 5

typedef int element;
typedef struct { // 큐 타입
    int front;
    int rear;
    element data[MAX_QUEUE_SIZE];
} LinearQueue;

void error(char *message) // 오류함수
{
    fprintf(stderr, "%s\n", message);
    exit(1);
}

void init_queue(LinearQueue *q)
{
    q->rear = q->front = -1;
}
```

```
void queue_print(LinearQueue *q)
{
    for (int i = 0; i < MAX_QUEUE_SIZE; i++) {
        if (i <= q->front || i > q->rear) printf(" | ");
        else printf("%d | ", q->data[i]);
    }
    printf("\n");
}

int is_full(LinearQueue *q)
{
    if (q->rear == MAX_QUEUE_SIZE - 1) return 1;
    else return 0;
}

int is_empty(LinearQueue *q)
{
    if (q->front == q->rear) return 1;
    else return 0;
}
```



```

void enqueue(LinearQueue *q, element item)
{
    if (is_full(q)) {
        error("큐가 포화상태입니다.");
        return;
    }
    q->data[++(q->rear)] = item;
}

int dequeue(LinearQueue *q)
{
    if (is_empty(q)) {
        error("큐가 공백상태입니다.");
        return -1;
    }
    int item = q->data[++(q->front)];
    return item;
}

```

```

int main(void)
{
    int item = 0;
    LinearQueue q;
    init_queue(&q);

    enqueue(&q, 10); queue_print(&q);
    enqueue(&q, 20); queue_print(&q);
    enqueue(&q, 30); queue_print(&q);

    item = dequeue(&q); queue_print(&q);
    item = dequeue(&q); queue_print(&q);
    item = dequeue(&q); queue_print(&q);
    return 0;
}

```

원형 큐

```

#include <stdio.h>
#include <stdlib.h>
#define MAX_QUEUE_SIZE 5

typedef int element;
typedef struct {           // 큐 타입
    int front;
    int rear;
    element data[MAX_QUEUE_SIZE];
} CircularQueue;

void error(char *message) // 오류함수
{
    fprintf(stderr, "%s\n", message);
    exit(1);
}

void init_queue(CircularQueue *q)
{
    q->rear = q->front = 0;
}

```

```

void queue_print(CircularQueue *q)
{
    printf("QUEUE(front=%d rear=%d) = ", q->front, q->rear);
    if (!is_empty(q)) {
        int i = q->front;
        do {
            i = (i + 1) % (MAX_QUEUE_SIZE);
            printf("%d | ", q->data[i]);
            if (i == q->rear)
                break;
        } while (i != q->front);
    }
    printf("\n");
}

int is_full(CircularQueue *q)
{
    return ((q->rear + 1) % MAX_QUEUE_SIZE == q->front);
}

int is_empty(CircularQueue *q)
{
    return (q->front == q->rear)
}

```

```

// 삽입 함수
void enqueue(CircularQueue *q, element item)
{
    if (is_full(q))
        error("큐가 포화상태입니다.");
    q->rear = (q->rear + 1) % MAX_QUEUE_SIZE;
    q->data[q->rear] = item;
}

// 삭제 함수
element dequeue(CircularQueue *q)
{
    if (is_empty(q))
        error("큐가 공백상태입니다.");
    q->front = (q->front + 1) % MAX_QUEUE_SIZE;
    return q->data[q->front];
}

```

```

// 삽입 함수
void enqueue(CircularQueue *q, element item)
{
    if (is_full(q))
        error("큐가 포화상태입니다.");
    q->rear = (q->rear + 1) % MAX_QUEUE_SIZE;
    q->data[q->rear] = item;
}

// 삭제 함수
element dequeue(CircularQueue *q)
{
    if (is_empty(q))
        error("큐가 공백상태입니다.");
    q->front = (q->front + 1) % MAX_QUEUE_SIZE;
    return q->data[q->front];
}

```



```

int main(void)
{
    CircularQueue queue;
    int element;
    init_queue(&queue);
    printf("--데이터 추가 단계--\n");
    while (!is_full(&queue))
    {
        printf("정수를 입력하시오: ");
        scanf("%d", &element);
        enqueue(&queue, element);
        queue_print(&queue);
    }
    printf("큐는 포화상태입니다.\n\n");

    printf("--데이터 삭제 단계--\n");
    while (!is_empty(&queue))
    {
        element = dequeue(&queue);
        printf("꺼내진 정수: %d \n", element);
        queue_print(&queue);
    }
    printf("큐는 공백상태입니다.\n");
    return 0;
}

```

□ 새로운 요소 e를 front에 삽입하는 add_front(e) 연산

알고리즘 4.7 원형 덩의 전단 삽입 연산

```

01 add_front(e)
02     if is_full() :
03         error "overflow"
04     else :
05         data[front] ← e
06         front ← (front-1+MAX_SIZE) % MAX_SIZE

```

□ rear 요소를 리턴하고 삭제하는 delete rear() 연산

알고리즘 4.8 원형 덩의 후단 삭제 연산

```

01 delete_rear()
02     if is_empty() :
03         error "underflow"
04     else :
05         prev ← rear
06         rear ← (rear - 1 + MAX_SIZE) % MAX_SIZE
07         return data[prev];

```

DEQUE

```

#include <stdio.h>
#include <stdlib.h>
#define MAX_QUEUE_SIZE 5

typedef int element;
typedef struct {
    int front;
    int rear;
    element data[MAX_QUEUE_SIZE];
} CircularDeque;

void error(char *message) // 오류함수
{
    fprintf(stderr, "%s\n", message);
    exit(1);
}

// 초기화 함수: 큐의 초기화 함수 (init_queue()) 함수와 동일
void init_deque(CircularDeque *q)
{
    q->rear = q->front = 0;
}

// 후단 삽입 함수: 큐의 enqueue() 함수와 동일
void add_rear(CircularDeque *q, element item)
{
    if (is_full(q))
        error("큐가 포화상태입니다.");
    q->rear = (q->rear + 1) % MAX_QUEUE_SIZE;
    q->data[q->rear] = item;
}

// 전단 삭제 함수: 큐의 dequeue() 함수와 동일
element delete_front(CircularDeque *q)
{
    if (is_empty(q))
        error("큐가 공백상태입니다.");
    q->front = (q->front + 1) % MAX_QUEUE_SIZE;
    return q->data[q->front];
}

```

```

void deque_print(CircularDeque *q)
{
    printf("DEQUE(front=%d rear=%d) = ", q->front, q->rear);
    if (!is_empty(q)) {
        int i = q->front;
        do {
            i = (i + 1) % (MAX_QUEUE_SIZE);
            printf("%d | ", q->data[i]);
            if (i == q->rear)
                break;
        } while (i != q->front);
    }
    printf("\n");
}

int is_full(CircularDeque *q) // 큐와 동일
{
    return ((q->rear + 1) % MAX_QUEUE_SIZE == q->front);
}

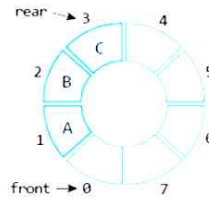
int is_empty(CircularDeque *q) // 큐와 동일
{
    return (q->front == q->rear)
}

// 후단 삽입 함수: 큐의 enqueue() 함수와 동일
void add_rear(CircularDeque *q, element item)
{
    if (is_full(q))
        error("큐가 포화상태입니다.");
    q->rear = (q->rear + 1) % MAX_QUEUE_SIZE;
    q->data[q->rear] = item;
}

// 전단 삭제 함수: 큐의 dequeue() 함수와 동일
element delete_front(CircularDeque *q)
{
    if (is_empty(q))
        error("큐가 공백상태입니다.");
    q->front = (q->front + 1) % MAX_QUEUE_SIZE;
    return q->data[q->front];
}

```

```
// 전단 참조 함수: 큐의 peek() 함수와 동일
element get_front(CircularDeque *q)
{
    if (is_empty(q))
        error( " 큐가 공백상태입니다 " );
    return q->data[(q->front+1) % MAX_QUEUE_SIZE];
}
```

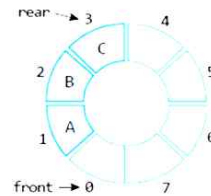


// 큐에 없는 새로운 함수

```
void add_front(CircularDeque *q, element item)
{
    if (is_full(q))
        error("큐가 포화상태입니다");
    q->data[q->front] = item;
    q->front = (q->front - 1 + MAX_QUEUE_SIZE) % MAX_QUEUE_SIZE;
}
```

// 큐에 없는 새로운 함수

```
element delete_rear(CircularDeque *q)
{
    int prev = q->rear;
    if (is_empty(q))
        error("큐가 공백상태입니다");
    q->rear = (q->rear - 1 + MAX_QUEUE_SIZE) % MAX_QUEUE_SIZE;
    return q->data[prev];
}
```



// 큐에 없는 새로운 함수

```
element get_rear(CircularDeque *q)
{
    if (is_empty(q))
        error("큐가 공백상태입니다");
    return q->data[q->rear];
}
```

```
int main(void)
{
    CircularDeque queue;

    init_deque(&queue);
    for (int i = 0; i < 3; i++) {
        add_front(&queue, i);
        deque_print(&queue);
    }
    for (int i = 0; i < 3; i++) {
        delete_rear(&queue);
        deque_print(&queue);
    }
    return 0;
}
```